

File Storage

1. Overview

File storage stores data as **files** organized inside **directories/folders** in a hierarchical structure.

Example:

None

root/

├─ users/

| └─ profile1.jpg

├─ docs/

| └─ report.pdf

└─ logs/

└─ app.log

It is one of the oldest and most widely used storage models.

2. Key Idea

You can think of file storage as a simple key-value system:

None

`key = file path`

`value = file content`

Example:

None

`/docs/report.pdf → PDF bytes`

`/images/a.png → image bytes`

`/logs/app.log → text data`

3. Core Components

Component	Meaning
File	Unit of stored data
Directory / Folder	Container for files

Path	Address/location
Metadata	Size, owner, timestamps, permissions

4. Example Paths

None

`/home/user/photo.jpg`

`/var/log/nginx.log`

`/app/uploads/video.mp4`

Each path uniquely identifies stored content.

5. Common Types of File Storage

A. Local File System

Stored on one machine disk.

Examples:

- Linux ext4
- NTFS
- APFS

B. Network File Storage

Shared across machines.

Examples:

- NFS
- SMB

C. Distributed File Systems

Large scale across clusters.

Examples:

- Hadoop HDFS
- Google File System

6. Why File Storage Is Popular

Simple to understand
Natural for images/videos/docs
Human-readable hierarchy
OS-level support
Easy backup and copy
Good for large binary files

7. Best Use Cases

Use file storage for:

- Images
- Videos
- PDFs
- Logs
- Documents
- Static website assets
- Backup archives
- ML model files

8. File Storage vs Database

Feature	File Storage	Database
Large binary files	Excellent	Moderate
Structured queries	Weak	Excellent
Transactions	Limited	Strong

Hierarchical folders	Native	No
Metadata search	Limited	Stronger

9. File Storage in Web Apps

Typical pattern:

None

Upload image

→ Save file in storage

→ Save path/URL in database

Example:

None

```
users.avatar_path = /images/u101.png
```

10. Cloud File/Object Alternatives

Modern systems often use object storage instead of local files:

- Amazon Web Services Amazon S3
- Google Cloud Google Cloud Storage
- Microsoft Azure Blob Storage

(These are object stores, not classic file systems, but commonly used for similar needs.)

11. File Metadata

Each file usually has:

None

name

size

created_at

modified_at

permissions

owner

type

12. Permissions / Security

Important controls:

- Read / write / execute
- User/group access
- Encryption at rest
- Backup policies
- Audit logs

13. Performance Considerations

Depends on:

- SSD vs HDD
- Network latency
- File size
- Concurrent access
- Caching
- Filesystem type

14. Challenges at Scale

Millions of small files
Shared access concurrency
Backup complexity
Metadata bottlenecks
Cross-region replication

15. Distributed File Systems

Used for big data.

Example:

Hadoop HDFS stores files across many nodes with replication.

Good for:

- ETL
- Analytics
- Batch processing

16. Interview Answer Example

Q: Where would you store user-uploaded videos?

I'd store large video files in file/object storage and keep metadata such as owner, path, and status in a database.

17. File Storage vs Key-Value

Feature	Key-Value DB	File Storage
Key lookup	Excellent	Path based
Binary large files	Moderate	Excellent
Querying metadata	Limited	OS/basic

Folder hierarchy	No	Yes
------------------	----	-----

18. Short Revision Notes

None

File Storage:

- Files + folders
- Path acts like key
- Good for images/videos/docs/logs

Examples:

Local FS, NFS, HDFS

19. One-Line Summary

File storage keeps data as files organized in folders, making it ideal for documents, media, logs, and large binary content where hierarchical path-based access is useful.